Quo Vadis Frontend?

Tymoteusz Huszcza

spoko tło, nie przyzwyczajać się ^

Przód koniec?

Kto to jest Frontend?

Co robi Frontend?

Jak robi Frontend?

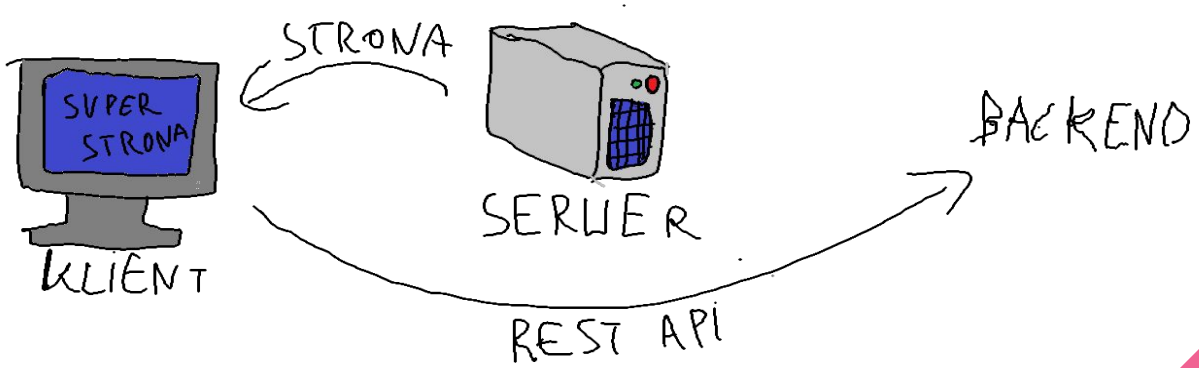


Kto...?

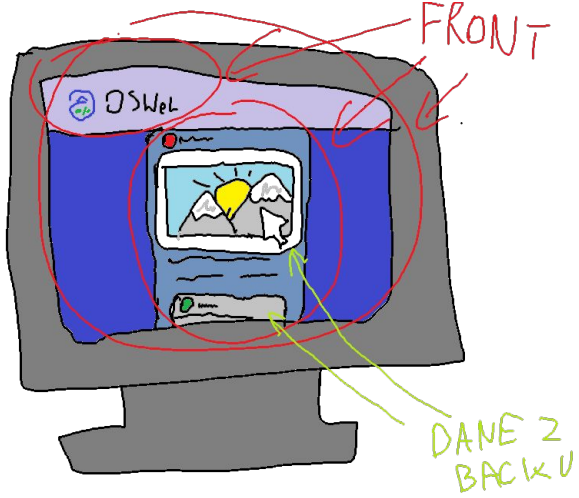
Frontend to warstwa aplikacji, odpowiadająca za wyświetlanie zawartości naszej aplikacji. Jest głównym holem sklepowym naszej aplikacji.

JEST STRONĄ

Jest widoczna dla klienta.



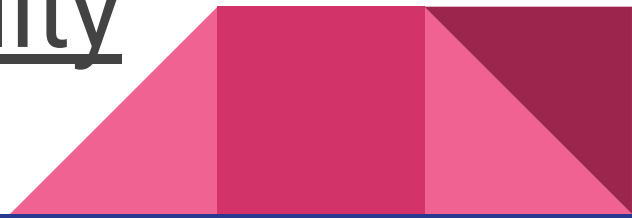
Co…?



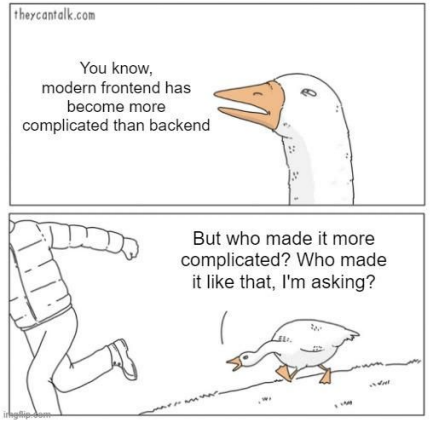
Ma za zadanie, wyświetlić całą zawartość naszej aplikacji, w jak najlepszy sposób. Odpowiada za wszystko co widzimy, od losowego przycisku, przez filmik, po wiadomości wysyłane prowadzącemu (ze zwolnieniem lekarskim przez chorobę alkoholową).

Jak...?

HTML, CSS, JS, UX, UI, SSG, DOM, EC6, RWD, SPA, PWA, SEO, Frameworki Frontendowe, TypeScript, State Management, CSS Preprocessing, Accessibility



HTML, CSS, SSG, SMG, SEO...?



HTML - znaczniki struktury strony
CSS - arkusze stylów strony
JavaScript - język programowania najczęściej używany na froncie, odpowiadający za logikę
UX (User Experience) - to jak przyjemnie i intuicyjnie używa się strony
UI (User Interface) - to jak ładna jest strona
SSG (Static Site Generation) - serwowanie gotowych, statycznych stron użytkownikom bez generowania ich na każde żądanie.
SMG (Semi-machine gun) - karabin półautomatyczny. idk do czego wam we frontendzie
DOM (Document Object Model) - obiektowa reprezentacja HTML
ES6 (ECMAScript 2015) - przełomowa wersja JS która wprowadza sporo fajnych, obecnie używanych funkcjonalności
RWD (Responsive Web Design) - dostosowywanie stron do dowolnej wielkości ekranu
SPA (Single Page Application) - pojedyncze ładowanie strony, które później zmienia wszystko bez przeładowania dokumentu
PWA (Progressive Web App) - stronka jako aplikacja na telefon (często z dodatkowymi funkcjami, np. działanie offline)
SEO (Search Engine Optimization) - optymalizacja i dobre praktyki pod wyświetlanie strony w wyszukiwarkach
Frameworki Frontendowe - biblioteki i narzędzia (zazwyczaj oparte na JS/TS) do budowy nowoczesnych stron, obecnie najpopularniejsze: React, Angular, Vue
TypeScript - nakładka/rozszerzenie do JS która dodaje typowanie do zmiennych
State Management - zarządzanie danymi, zajmujące się przechowywaniem, aktualizowaniem i udostępnianiem danych potrzebnych różnym częściom interfejsu
CSS Preprocessing - rozszerzenia do starego, dobrego CSS dodające opcje programistyczne, np. pentelki
Accessibility - projektowanie stron z myślą o osobach niepełnosprawnych

Po co...?

Tworzenie frontendu i backendu jako niezależnych części aplikacji, komunikujących się przez protokół HTTP, **pozwala na niezależny rozwój obu warstw, co znacząco przyspiesza proces tworzenia oprogramowania**.
Programiści frontendowi mogą pracować na przykładowych danych (np. z plików .json) bez konieczności czekania na gotowe API, podczas gdy backendowcy mogą testować i dokumentować endpointy w dowolnym czasie.

Kluczowe korzyści takiego podejścia to:

- **Łatwiejsze skalowanie i elastyczność**: Jeden backend może obsługiwać wiele różnych interfejsów (strony www, aplikacje mobilne, urządzenia IoT) bez konieczności przepisywania logiki serwerowej.
- **Specjalizacja zespołu**: Programiści mogą pracować w swoich ulubionych technologiach (np. Java czy Python dla backendu, React czy Angular dla frontendu) i edytorach, co zmniejsza ryzyko błędów wynikających z braku doświadczenia w danej dziedzinie.
- **Szybsze wdrażanie i utrzymanie**: W przypadku zmiany technologii w jednej warstwie (np. frameworku widoku) nie jest konieczne przepisywanie całego projektu, a łatwiejsze jest wdrożenie ciągłego dostarczania (continuous delivery).

Podział ten eliminuje problemy typowe dla dużych, monolitycznych projektów, takich jak długie czasy ładowania i konieczność czekania na współpracowników, a jednocześnie pozwala na równoległą pracę zespołów nad różnymi częściami systemu.

Odpowiedź wygenerowana przez AI. Prosimy zweryfikuj najważniejsze fakty.

Obecnie najpopularniejsze technologie i kilka słów o nich

Obecnie najpopularniejszymi frameworkami są kolejno: React, Angular, Vue.

Oczywiście, różnią się one dosyć strukturą, szybkością, wielkością, sposobem pisania kodu, itd.

- React (od Mety/Facebooka🤮) - najpopularniejszy (technicznie nie-do-końca) framework. Technicznie to tylko "biblioteka" do budowy UI, a nie pełny framework, więc dużo narzędzi trzeba dobierać samemu. Bardzo elastyczny. Wykorzystuje tzw. Virtual DOM do sprawdzania zmian i aktualizacji rzeczywisty DOM.
- Angular (od Google) - pełnoprawny framework. Dostarcza wszystko "w pudełku". Jest bardzo ustrukturyzowany i narzuca swój konkretny sposób pisania kodu. Uwielbiany przez wielkie korporacje. Z góry wymusza używanie TypeScriptu.
- Vue.js - prosty do nauki na start, lekki, ale też bardzo potężny jak się go dobrze opanuje. Częsty wybór do mniejszych i średnich projektów.

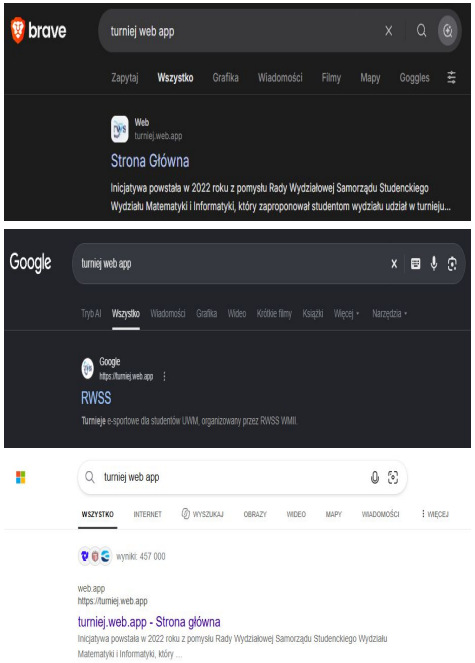


WebSockets API… znów…

Otwieramy kanał komunikacji front - back. Przy pomocy WebSocket'ów, wygodnie możemy odbierać sygnał od backendu że coś się zmieniło i warto odświeżyć dane lub zrobić coś innego. Też możemy przekazać dla backendu że coś na froncie się zmieniło. Nie zawsze się to opłaca, ale jest to fajne kiedy chcemy mieć dane real-time, jak przy chatach (np. Facebook Messenger)



SEO, czyli jak Google psuje wyszukiwania



To jak tworzymy stronę, wpływa później na to jak jest pozycjonowana i wyświetlana w przeglądarce, po wyszukaniu. Jednak są też ogólne zasady, praktyki i inne triki które trochę nam w tym pomagają, takie jak metadane strony. Dlatego stosujemy:

- odpowiednie tagi, jak <article>, <header> czy <nav>
- metatagi i OpenGraph czyli <title>, <meta> z odpowiednimi rzeczami, itd.
- SSR/SSG, do zwracania faktycznie uzupełnionej strony dla bota indeksującego stronę.

Podstawowe zabezpieczenia frontendu

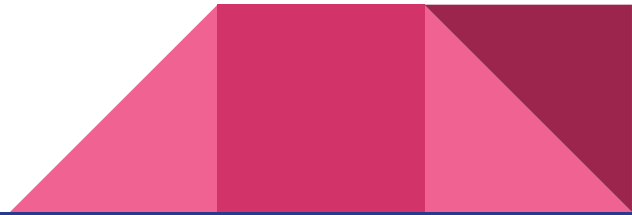
XSS (Cross-Site Scripting) - sanityzowanie i filtrowanie tego, co user wpisuje i co dostajemy, żeby ktoś nam nie wstrzyknął JS

Bezpieczne trzymanie kluczy - JWT najlepiej trzymać w ciasteczkach z odpowiednimi flagami, a nie w widocznym dla każdego localStorage

CORS - „szajs blokujący nieupoważnione domeny”, ale tutaj użyteczny, patrzy czy strona może dostawać odpowiedzi z innego originu (taka yandere)

CSRF (Cross-Site Request Forgery) - pilnujemy, żeby ktoś nie podpiął pod nasz przycisk na stronie, akcji na innym serwerze, wykorzystując np. tokeny Anti-CSRF

Błędna kontrola dostępu - użytkownik powinien widzieć i robić tylko to, do czego ma uprawnienia



Dobre praktyki

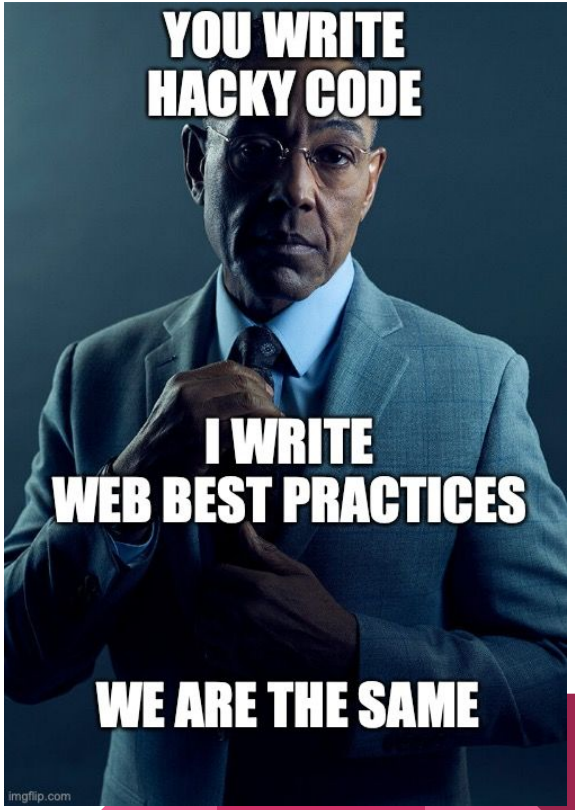
Komponentowość - pamiętaj programisto młody, pisanie tego samego komponentu 50 razy jest k*rwa głupie, po prostu zrób raz i importuj

Walidacja - pamiętaj programisto młody, użytkownik z założenia jest głupi, więc warto od razu go o tym uświadamiać (no i by nie mógł wysyłać niepoprawnych danych)

Optymalizacja - pamiętaj informatyku młody, nie ładujemy użytkownikowi do przeglądarki 100 MB obrazków i kodu od razu na start, a pobieramy na bieżąco co aktualnie widzi (Lazy loading)

Clean Code - pamiętaj informatyku młody, tutaj też, SOLID DRY KISS

Minimalizm - pamiętaj frontendowcu młody, warto nie robić dużego i długiego kodu



Pytania? nie? super.

Autor: Tymoteusz Huszcza

Źródła:

- Google
- Google for devs
- Roadmap.sh
- Medium
- Wikipedia

